Integration of GenAI with Traditional Software Testing Frameworks

Automated Software Engineering

Bruno Silva
University of Coimbra
2021232021

Diogo Honório
University of Coimbra
2021232043

Rodrigo Borralho
University of Coimbra
2021233455

November 2025

Abstract

Generative AI (GenAI) has emerged as a revolutionary force in software testing, offering unprecedented capabilities to automate and enhance traditional testing frameworks. This paper consists on the integration of GenAI into software testing practices, analyzing its impact across the software development life cycle (SDLC). We examine how GenAI, particularly Large Language Models (LLMs), addresses critical testing tasks including unit test case generation, test oracle generation and system test input generation. Our analysis revealed substantial benefits: up to 30% reduction in test case generation time, 95% test coverage compared to 85% with conventional approaches and 20% higher defect detection rates. The paper systematically compares traditional testing methodologies, including manual versus automated and black-box versus white-box approaches with GenAI-powered testing across multiple dimensions: speed, coverage, adaptability, efficiency, error detection, maintenance, complexity handling, and scalability. However, we also identify critical issues that require resolution,, including AI hallucinations, model drift, absence of continuous evaluation standards, lack of standardized metrics, limited human oversight and catastrophic forgetting. Based on these findings, we propose future insights on how GenAI could improve further on Testing and Evaluation (T&E) frameworks, development of AI-specific quality metrics, standardized protocols for risk management and thorough training programs for software testing engineers. This effort adds value to a better understanding of both the paradigm-shifting potential and possible risks of GenAI integration in software testing, providing a balanced perspective for developers and researchers traversing this fast-changing evolving landscape.

1 Introduction

Generative AI (GenAI) has been gaining significant ground in software development and testing. The main goal of this research is to explore the integration of Generative AI into software testing, highlighting its potential benefits, challenges, risks and future perspective and concerns.

1.1 Contextualization of the Theme

Software testing is the process of evaluating and verifying that a software product or application functions correctly, securely and efficiently according to its specific requirements [1]. Today, software testing is fully integrated into modern development practices, driven by Agile methodologies, DevOps and continuous integration/continuous delivery (CI/CD) pipelines. It starts during the design and planning phase and continues after deployment.

Furthermore, the emerge of Artificial Intelligence (AI) into software development led to complete a transformation of the development process by introducing tools and techniques that enhance productivity, accuracy and innovation [2]. AI, particularly generative AI (GenAI) and large language models (LLMs), streamline the development cycle

by automating key steps. By using natural language processing (NLP), AI can transform a simple prompt into a complete workflow, covering everything from idea generation and requirement gathering to coding and testing.

2 Background

2.1 Software Testing

In today's world, the consequences of software defects are more severe than ever, with the potential to damage a brand's reputation and lead to dissatisfied customers. In severe cases, a bug or defect can degrade interconnected systems or cause serious malfunctions.

For this reason, as previously said, software testing has become a crucial phase of the software development life cycle (SDLC), securing that applications meet both functional and non-functional requirements before deployment. Moreover, testing ensures that the software is reliable, performs efficiently, and delivers a consistent user experience under various conditions.

Software testing consists in various approaches and methodologies, each addressing different aspects of quality assurance. Among these, the most common classifications are manual vs. automated testing and black-box vs. white-box testing, which together provide complementary perspectives to ensure software correctness and robustness.

2.1.1 Manual versus automated testing

A. Manual Testing: Manual testing is the execution of test cases without the assistance of automated tools. Therefore, manual testing involves manually checking for defects by testing each function one by one to ensure they work as expected.

B. Automated Testing: Alternatively, automated testing is a technique where the tester writes scripts independently and uses suitable software or automation tools to test the software or application.

2.1.2 White Box Testing

White box testing is a software technique that involves testing the internal structure of the software application while having full access to the source code. The tester uses this knowledge to design test cases that can verify the correctness of the software at the code level.

2.1.3 Black Box Testing

On the other hand, black-box testing is an approach in which the tester disregards the internal structure and implementation details of the software, focusing instead on verifying its functionality according to the given specifications and requirements. This testing technique is typically categorized into two main types: functional testing, which evaluates what the system does, and non-functional testing, which assesses how well the system performs under certain conditions.

2.1.4 Levels of Testing

Regarding the Black Box technique, functional software testing is structured into four hierarchical levels within the software development life cycle, with each level targeting distinct aspects and functionalities of the application:

- Unit Testing: Validates if each individual software unit functions as expected. A unit represents the smallest testable component of an application.
- Integration testing: Ensures that software components or functions work together effectively.
- System testing: Evaluates the end-to-end performance of the entire system. This phase includes aspects of functional testing, nonfunctional testing, interface testing, stress testing and recovery testing.
- Acceptance testing: Verifies whether the whole system works as intended, ensuring that the requirements of the users are fulfilled before its delivery and that the software works correctly in the user's working environment.

2.2 Generative AI

Earlier versions of GenAI consisted on sophisticated machine learning models called deep learning models algorithms - such as Recurrent Neural Networks (RNNs), Generative Adversarial Networks (GANs) and Convolutional Neural Networks

(CNNs) - that simulate the learning and decision-making processes of the human brain [3].

In 2017, transformer models were introduced, marking a pivotal moment in GenAI history as they addressed many of the earlier models' limitations. These transformers employed mechanisms such as self-attention and feed-forward neural networks to revolutionized sequence modeling. Such innovation enabled parallel processing of data which significantly improved the scalability and efficiency of the models in handling long-range dependencies [4].

Built on the Transformer architecture, LLMs such as Generative Pre-Trained Transformer (GPT) and Bidirectional Encoder Representations and Transformers (BERT) emerged between 2018 and 2019. These models are based on extensive pre-training over diverse and large-scale datasets that enabled generating coherent and contextually relevant text. While GPT excelled in text generation, BERT introduced bidirectional encoding for improved understanding of the text context [4].

3 GenAI in Software Testing

3.1 The Rapid Adoption of GenAI in Software Testing

According to "Software Testing with Large Language Models: Survey, Landscape, and Vision" [12], the number of publications on Generative AI in software testing has skyrocketed, growing from just 2 in 2021 to 82 by 2023. This surge is largely attributed to the release of ChatGPT in 2022, which rapidly expanded public and industry interest in the field. By July 2025, ChatGPT had been used weekly by more than 700 million users [13], establishing it as one of the most popular Generative AI tools and demonstrating how this technology has become an essential part of modern software development and testing practices, influencing both research and industry adoption.

3.2 Analysis of GenAI Contributions Across Software Testing Tasks

In this section, we analyze the distribution of each testing task, followed by further analysis for each task. LLMs have demonstrated effectively used across mid to late stages of the SDLC.

During the test case preparation phase, LLMs have been employed with tasks such as generating unit tests cases, test oracle generation and system test input generation. These tasks are crucial in the mid-stage of software testing, helping identify defects and preventing the propagation of errors into later development steps.

Furthermore, in subsequent phases—particularly test reporting, bug reporting, and remediation—LLMs have been utilized for tasks encompassing bug analysis, debugging, and automated repair. These tasks assume critical importance during the latter stages of the testing lifecycle, when software defects must be systematically resolved to ensure product readiness for deployment [12].

3.2.1 Unit Test Case Generation

Unit test case generation is crucial for automating checks on individual software components and ensuring their correctness.

3.2.2 Test Oracle Generation

A Test Oracle is a mechanism in software testing that determines if a software's output is correct by comparing the actual results to expected results.

3.2.3 System Test Input Generation

System Test Input Generation helps to gather inputs that cover the desired scenarios and to cover all requires of unit test cases generated.

3.3 Key Benefits

3.3.1 Automate and Accelerate Test Case Generation

The traditional process of manually generating test cases is both time-consuming and resource-intensive. The implementation of GenAI in testing has enabled the automatic generation of relevant and comprehensive test cases that accurately reflect real-world user stories and functional requirements.[5] This advancement allows testing teams to complete the testing process at

a much faster pace, facilitating more rapid iterations and enabling quicker feedback on development changes.[10] Studies have shown a reduction of up to 30% in test case generation and execution time, highlighting the benefits of using GenAI in testing compared to traditional methods.[11]

3.3.2 Reduce of manual work

The integration of GenAI into testing has significantly reduced manual effort by automating the creation of test cases. Its ability to dynamically adapt to changes minimizes the need for human intervention [11], allowing to suggest corrective actions and optimizations to further decrease manual involvement throughout the testing process [5].

3.3.3 Continuous Testing and Monitoring

Additionally, GenAI plays an important role in continuous testing frameworks, where it is capable of generating tests that are updated with each new code change or feature addition.[10] This enables testing to extend beyond deployment, allowing it to reflect the current system state, detect anomalies, identify the root causes of test failures, and monitor software performance throughout the operational lifecycle [5].

3.3.4 Defect Detection

Moreover, GenAI is responsible for identifying discrepancies between the actual and expected results, referred to as defects. These defects enable the early recognition of irregularities and potential issues that traditional validation methods might overlook,[11] and the insights gained can be leveraged to enhance future test case generation[10]. Studies have shown that the integration of GenAI in testing has achieved a 20% higher detection rate compared to traditional methods[11].

3.3.5 Early Feedback

The use of GenAI for rapid prototyping enables developers and end users to provide early feedback, minimizing potential misunderstandings of user needs and supporting more accurate system design from the early stages[10]. Although, while GenAI can generate tests autonomously, human testers can provide valuable feedback to review and fine-tune the test generation process.

3.3.6 Test Coverage

One of the main features of GenAI testing is its ability to identify and generate a diverse range of test cases [10]. The analysis of the studies showed that, in terms of test coverage, GenAI methods achieved 95% coverage, surpassing the 85% attained by conventional approaches[11]. This improvement is due to GenAI's capability to generate a broad variety of test cases, including edge cases and boundary conditions that are often overlooked in traditional testing [10].

3.3.7 Minimizes Human Error

Manual test generation has significant potential for human error across various factors, and such errors can result in incomplete or inaccurate test coverage, potentially allowing critical defects to go unnoticed. GenAI has the capability to reduce these risks because it operates based on learned patterns and established best practices. This enables it to produce more consistent outcomes and helps prevent the types of errors commonly introduced in manual testing.[10]

3.4 Challenges and Considerations

3.4.1 Hallucinations and Inaccuracy

There is a risk of hallucinations, where GenAI perceives wrong information, creating outputs that are inaccurate. This presents a high risk since if many processes are integrated with GenAI then a waterfall effect can occur, where the errors are propagated across dependent processes[5][7].

3.4.2 Model Drift

Model drift refers to the degradation of machine learning model performance due to changes in data or in the relationships between input and output variables. This presents a high risk since, if not early detected, it increases the harm to operations overtime, to avoid it, a constant monitoring and managing of the organization and AI model is required, which leads, for example, to resource management complexity[8].

3.4.3 Absence of Continuous Evaluation Standards

Since AI models can degrade over time, the risks discussed in this section might only be detected too late without continuous evaluation, including testing and evaluation (T&E) activities. Moreover, as AI technologies rapidly evolve, evaluation methods must also adapt to remain effective. Without systematic ongoing evaluation practices, organizations risk undetected performance degradation, operational failures, and unreliable outputs[5][9].

3.4.4 Lack of Standardized Evaluation Metrics

Traditional evaluation metrics often fail to reflect actual user success or workflow reliability, as such, user feedback from real deployments is required instead of relying on controlled testing environments[5][14].

3.4.5 Limited Human Oversight / Misuse Risk

As a consequence of the risks above, the integration of GenAI in many procedures, along with limited resources, time, and overtrust in AI, creates a challenge to maintaining adequate human oversight, both to ensure the AI model is properly running and not compromising results, and to make sure people are making proper use of it.

3.4.6 Catastrophic forgetting

This phenomenon occurs when the AI model forgets the data it was trained on, causing it to become potentially harmful[15].

4 Traditional vs. GenAI Testing

As its represented in Table 1, AI-driven test case generation allows for a more efficient, adaptive and scalable approach to software testing comparing to traditional methods, making it an essential tool for modern software development and testing. While traditional testing relies heavily on human effort and predefined scenarios, GenAI testing leverages machine learning to autonomously generate and execute test cases, ensuring broader coverage, predicting defects and continually learning from historical data (if its model allows for long-term memory support) [10].

5 Future of GenAI in Testing

Despite the gains of GenAI in testing, an important point that is required to highlight in the future is the need for investment in specialized GenAI Testing and Evaluation (T&E) research, mainly in the development of new metrics for AI qualities such as fairness, explainability, and uncertainty quantification [5]. This research would be essential, since the current GenAI systems still lack meaningful theoretical or empirical safety guaranties [6].

Another important area to take into account involves the creation of tools for automated issue detection. Although progress in this field is evident, given that a year and a half ago, the primary focus was still on the automation of test generation and coverage checking [6], significant challenges remain. Problems such as hallucinations and concept drift continue to affect model reliability, underscoring the need for further improvement [5].

A third important research direction is the exploration of formal and model-checking methods, along with the generation of adversarial test cases to stress-test model limits [5]. Finally, it is crucial to develop techniques for systems that learn and adapt over time after deployment, ensuring that evaluation and safety mechanisms evolve alongside the models themselves [5].

In addiction to these directions, future work should also aim to achieve full-automation in LLM-based solutions, enabling testers to focus on more creative tasks while maintaining reliable results. Optimization of prompt engineering for LLM-based solutions using advanced prompting strategies, such as few-shot learning and chain-of-thought techniques [12], should also be an aspect to take into account, as well as the adoption of semantic-based approaches in order to improve the accuracy of generated assertions by LLM-based solutions [4].

Equally important for the future of GenAI is training developers and test engineers in AI-based testing tools. Such training will allow professionals to gain a deeper understanding of the strengths and

limitations of AI systems, enabling them to evaluate effectiveness more accurately. By developing these skills, practitioners can prevent misuse of AI, interpret outputs with greater precision, and apply critical analysis when reviewing AI-generated results [5]. This is essential, as test engineers will continue to serve as the ultimate authority, ensuring that test outcomes remain reliable, safe, and secure [6].

Another important factor to consider is the development of standardized frameworks and protocols to manage the unprecedented risks associated with GenAI. There is a pressing need to establish T&E protocols that specifically address AI-related aspects of quality and risk, taking into account the type of AI system, its operational domain, and the level of human involvement in its deployment and oversight [5]. These standards should define clear testing requirements, including the minimum evaluations necessary before deployment, as well as checklists for ethical risk assessment during the T&E process. Furthermore, such standards must extend to ongoing T&E activities throughout the SDLC. This includes the implementation of policies for continuous performance monitoring, the collection of operational data post-deployment and the establishment of mechanisms to ensure periodic retraining and re-testing as the system's environment or requirements evolve [5].

Finally, it is essential to formalize standardized test suites and benchmarks for different classes of GenAI outputs, providing consistent baselines for performance comparison and improvement [5].

6 Discussion

GenAI is a developing technology with high potential and broad applicability, capable of driving the evolution of companies by solving complex problems, automating repetitive tasks, creating space for new activities, improving resource allocation, and increasing operational efficiency.

That said, due to its enormous potential, constant monitoring is necessary. If not properly supervised, it can cause significant losses, as when integrated with multiple systems, unsatisfactory outputs may affect the entire system, especially if detected late. In other words, GenAI must be implemented in a consistent and controlled manner, avoiding automatic trust in its outputs so that the reduction of human labor becomes a benefit rather than a risk leading to undesirable losses.

Despite these risks, the adoption of GenAI has become almost inevitable. Companies that do not integrate it may fall behind the competition, missing opportunities for efficiency, innovation, and resource optimization. On the other hand, those that choose not to use the technology, maintain full control over processes and avoid the risks associated with incorrect outputs or system failures, but remain subject to human errors, productivity limitations, and reduced responsiveness in dynamic environments. Thus, a delicate balance emerges between leveraging the potential of GenAI and managing the inherent risks of its implementation.

Feature	Traditional Testing	GenAI Testing
Speed	Slow, requires manual effort to create test cases and execute them.	Rapid, autonomous generation and execution of test cases.
Coverage	Limited to predefined test cases based on human intuition and requirements.	Broad and dynamic coverage, able to cover edge cases, boundary conditions and hidden scenarios not considered in traditional tests.
Adaptability	Requires frequent updates and manual intervention to adjust test cases when business logic or regulations change.	Self-learning and auto-updating, with AI continuously adapting to new data, requirements, and system changes.
Efficiency	High effort, repetitive, and time-consuming, often requiring additional resources to handle large test sets.	Automated and cost-effective, allowing teams to focus on execution and validation while AI handles the test generation.
Error Detection	Reactive approach: errors are identified after execution, often when it's too late to fix them in the current cycle.	Proactive and predictive; AI can predict high-risk areas and detect defects before they even occur in production.
Maintenance	Requires ongoing manual maintenance as applications evolve.	Minimal manual intervention needed for maintenance, as AI models self-learn and adapt.
Test Complexity	Limited by the tester's understanding and scope, often missing complex or rare scenarios.	AI-driven testing uncovers hidden paths and corner cases that might otherwise be missed.
Scalability	Difficult to scale with increasing complexity or size of the application, often requiring additional testers and resources.	Easily scalable with the growth of the application, as AI can handle larger test sets and more complex scenarios without extra manual effort.

Table 1: Tradicional vs GenAI Testing

References

[1] Susnjara, S., & Smalley, I. (2025, July 22). Software Testing. IBM. https://www.ibm.com/think/topics/software-testing

[2] Finio, M., & Downie, A. (2025, October 22). AI in Software Development. IBM. https://www.ibm.com/think/topics/ai-in-software-development

[3] Stryker, C., & Scapicchio, M. (2025, October 21). Generative AI. IBM. https://www.ibm.com/think/topics/generative-ai

[4] K. Almohsen , F. Albalooshi and J. A. Ammari (2025) "Exploring the Use of GenAI in Automating Software Testing". https://doi.org/10.1109/icmi65310.2025.11139839

[5] Freeman, L., Robert, J., & Wojton, H. (2025). The Impact of Generative AI on Test & Evaluation: Challenges and Opportunities. https://doi.org/10.1145/3696630.3728723

[6] Layman, L., & Vetter, R. (2024b). Generative artificial intelligence and the future of software testing. Computer, https://doi.org/10.1109/mc.2023.3306998

[7] Ibm. (2025, October 31). AI Hallucinations. https://www.ibm.com/think/topics/ai-hallucinations

[8] Holdsworth, J., Belcic, I., & Stryker, C. (2025, October 20). Model drift.

https://www.ibm.com/think/topics/model-drift

[9] Duranton, S. (2024, May 9). Beyond Accuracy: The changing landscape of AI evaluation. Forbes. https://www.forbes.com/sites/sylvainduranton/2024/03/14/beyond-accuracy-the-changing-landscape-of-ai-evaluation/

[10] Pareek, C. S. (2025). Autonomous test case generation using GenAI for life insurance applications. International Journal of Multidisciplinary Research and Growth Evaluation, https://doi.org/10.54660/.ijmrge.2025.6.2.414-424

[11] Ale, N. K. (2024). A generative AI framework for enhancing software test automation: design, implementation, and validation. International Journal of Science and Research (IJSR), https://doi.org/10.21275/sr24604032016

[12] Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., & Wang, Q. (2024). Software testing with large language models: survey, landscape, and vision. IEEE Transactions on Software Engineering, 50(4). https://doi.org/10.1109/tse.2024.3368208

[13] Chatterji, A., Cunningham, T., Deming, D., Hitzig, Z., Ong, C., Shan, C. Y., & Wadman, K. (2025). How people use ChatGPT. https://doi.org/10.3386/w34255

[14] Jinggeoffrey. (2025, September 19). The Evaluation Gap: Why AI Metrics Don't Predict Real-World Success. Medium. https://medium.com/@jinggeoffrey/the-evaluation-gap-why-ai-metrics-dont-predict-real-world-success-f525ae0a6f92

[15] Belcic, I., & Stryker, C. (2025, October 21). Catastrophic Forgetting. https://www.ibm.com/think/topics/catastrophic-forgetting